ACTIVITY 5

Database Maintenance and Monitoring

Monitoring Database Health, Maintaining Reliability, and Analyzing Performance

Course	COMP 018 – Database Administration	**Activity Type**	Group Activity
Database Platform	PostgreSQL	**Core Skill**	Monitoring, maintenance, and performance analysis

Activity Overview

Your group will monitor and maintain the PostgreSQL databases of the projects it set up in Activity 4, which are the Activity 3 projects of all the other groups, found through the class Google Sheets tracker. Each database should reflect the application setup from Activity 3 and the access-control configuration from Activity 4. For each project, your task is to assess its condition, inspect database activity and available logs, perform appropriate routine maintenance, and evaluate the performance of a selected query.

This activity emphasizes practical database administration. Students must collect evidence, interpret their observations, explain possible issues, and recommend appropriate actions. Do not make changes without understanding their purpose and potential impact. Results are documented in the group's PDF progress documentation, kept alongside its own project repository and recorded in the Google Sheets tracker.

I. Learning Objectives

By the end of this activity, students should be able to:

1. Inspect PostgreSQL database status, connections, sessions, and activity.
2. Examine available logs and identify potential errors or warning signs.
3. Perform appropriate routine maintenance operations and explain their purpose.
4. Analyze a query execution plan and identify possible performance concerns.
5. Relate monitoring and maintenance findings to the roles, accounts, and privileges set up in Activity 4.
6. Document observations, maintenance actions, test results, and recommendations.
7. Keep the work in the group's own repository, record its status in the class Google Sheets tracker, and submit a PDF progress document.

II. General Instructions

- Continue using the other groups' projects that you cloned and set up in Activities 3 and 4, together with the access-control configuration from Activity 4. Find each repository and its user roles documentation in the Google Sheets tracker.
- Use a local or instructor-approved test environment. Do not run potentially disruptive commands on production systems.
- Use an authorized database account from the Activity 4 access-control plan, choosing the account whose role fits each task (for example, a read-only or auditor account for inspection). Do not bypass the privileges established in Activity 4.
- Capture screenshots or command output as evidence and explain what each result means.

- AI tools may be used for explanations or troubleshooting, but your group must validate suggestions and explain the commands it runs.
- Never include real passwords, API keys, or other secrets in your report, repository, tracker, or PDF.
- Work through every other group's project listed in the tracker (not your own). If a project's database cannot be reached or set up, record the blocking problem and continue with the others.
- Keep your work in your group's own repository (for example, a db-maintenance/ folder with one subfolder per project). Do not modify any other group's repository; send recommendations to the owning group as feedback.

III. Activity Tasks

Part A – Database Health and Activity Inspection

For each project, connect to its PostgreSQL database and collect evidence about its current state. Use psql or an approved database administration tool.

Inspect and document the following:

- PostgreSQL version and the name of the database being examined.
- Database size and the number of tables or other relevant objects.
- Current sessions and connections, including active or idle sessions where visible.
- Currently running queries and their duration, where permissions allow.
- Available evidence of failed connections, errors, or unusual behavior.
- The Activity 4 roles and login accounts present in the database, confirming that they still match the access-control plan (record any difference).
- The account used for each check and the application role it serves.

Possible commands and queries (adapt them to your environment):

```
SELECT version();
SELECT current_database();
SELECT pg_size_pretty(pg_database_size(current_database()));

SELECT datname, numbackends
FROM pg_stat_database
WHERE datname = current_database();

SELECT pid, usename, state, wait_event_type, wait_event,
       query_start, query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY query_start NULLS LAST;
```

> Note: Monitoring views may show limited information depending on the account's privileges. Record permission limitations rather than attempting to bypass them.

Part B – Logs and Issue Identification

Inspect the PostgreSQL logs available in your configured environment. Log location and access depend on the installation and configuration; logs may be viewed through the server's configured log destination or an administrator-approved service/tool.

Look for evidence such as:

- Failed login or connection attempts.
- SQL errors or application-generated database errors.
- Long-running queries or signs of blocked activity, if logged or observable.

- Warnings or repeated errors that may affect reliability.
- Permission-denied errors that may come from the Activity 4 restrictions, and whether they are expected or show a missing privilege.

For each finding, record the evidence, a plausible explanation, the impact if left unresolved, and a recommended next step. If no relevant issue appears in the available logs, state what you checked and what limitations applied. Do not invent incidents.

Part C – Routine Maintenance

Perform and document suitable maintenance operations in each test database. Before running a command, explain its purpose and confirm that it is appropriate for the environment.

Required maintenance work:

- Review table and index objects and identify whether statistics or maintenance may be needed.
- Run ANALYZE on an appropriate table or the database to refresh planner statistics.
- Run VACUUM on an appropriate table, or explain and demonstrate the suitable VACUUM approach for your environment.
- Confirm that the operation completed and document the result.

Example commands:

```
ANALYZE;
VACUUM (VERBOSE, ANALYZE) public.example_table;
```

> Replace public.example_table with a real table in each project's database. Run VACUUM as a standalone command, not inside a transaction block. Do not use VACUUM FULL for this activity unless your instructor explicitly approves it; it can require an exclusive lock and additional disk space. Maintenance needs depend on database activity, table size, autovacuum configuration, and the environment. Use an account whose privileges allow the operation; do not switch to a superuser merely to make a command work. If the Activity 4 privileges do not allow maintenance, record the limitation and recommend a suitable maintenance role instead.

Part D – Query Performance Analysis

For each project, select one meaningful SELECT query used by the application, ideally one run by a documented application role (see its user roles documentation). Capture its execution plan, interpret the main operations, and identify at least one potential improvement or explain why the plan appears reasonable.

Use EXPLAIN first. EXPLAIN ANALYZE executes the query, so use it only for a safe SELECT query in the test environment.

```
EXPLAIN
SELECT *
FROM public.example_table
WHERE example_column = 'sample_value';

-- Optional, only for a safe SELECT query in the test environment:
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM public.example_table
WHERE example_column = 'sample_value';
```

Discuss the scan type (for example, sequential scan or index scan), estimated versus actual rows if EXPLAIN ANALYZE was used, execution time where available, and whether an index or query rewrite may help. Do not create an index merely to satisfy the activity; justify any proposed change and consider the write/storage cost of indexes.

Part E – Findings and Maintenance Report

Prepare a concise report for each project that includes:

- Environment and database inspected, with the accounts used and the Activity 4 roles they represent.
- Health and activity checks performed, with evidence.
- Log findings or a statement that no relevant issue was observed in the available logs.
- Maintenance commands run, why they were selected, and their outcomes.
- Query plan, interpretation, and any justified performance recommendation.
- Limitations encountered, including permission or logging limitations.
- Prioritized recommendations: immediate action, follow-up action, or no action required.

Combine the project reports into the group's PDF progress documentation (see Part F).

Part F – Repository, Tracker, and Progress Documentation

As in Activities 3 and 4, there is no centralized repository. Each group keeps its Activity 5 work in its own project repository and records its status in the class Google Sheets tracker. The group then submits a PDF that documents all of its progress.

Step 1 – Keep your work in your own repository

1. Create a folder such as db-maintenance/ in your group's own repository, with one subfolder per project (for example, db-maintenance/group-03/) containing that project's maintenance script, query plan evidence, log notes, and report.
2. Use meaningful commits so the repository history shows how the monitoring and maintenance work progressed.
3. Do not commit passwords or secrets. Use placeholders in scripts and keep real credentials in local configuration.
4. Do not change any other group's repository. Send findings and recommendations to each owning group as feedback instead.

Step 2 – Update the Google Sheets tracker

Keep your group's Activity 3 row up to date. In addition, add a tab named Activity 5 Log to the tracker with one row for every project your group monitors, using the columns below. Edit only your own group's rows.

Activity 5 Log column	What to enter
Project	Group number and project name of the database monitored.
Database inspected	Database name and PostgreSQL version.
Account and role used	The Activity 4 account used and the application role it serves.
Health check result	Healthy, Issues found, or Not reachable, with a short note.
Maintenance performed	ANALYZE, VACUUM, or other action, with the date.
Query analyzed	Short description of the query and the scan type observed.
Recommendation priority	Immediate action, Follow-up action, or No action required.
Evidence folder link	Link to that project's subfolder in your own repository.
Feedback sent to owners	Yes or No, with the date feedback was sent to the owning group.
PDF progress document	Link to your group's PDF progress documentation (same link on every row).
Last updated	Date of the most recent update by your group.

Step 3 – Submit the PDF progress documentation

Continue the PDF progress document from Activities 3 and 4 by adding the sections below, or submit a separate PDF for this activity. Submit it together with the link to your repository.

Section	Contents
Cover and links	Group number, members, repository link, and tracker link.
Projects monitored	Table of every project, its repository link, database name, and the accounts and roles used.
Health and activity evidence	For each project, command output or screenshots with interpretation.
Log findings	Evidence, explanation, impact, and next step for each finding, or what was checked if nothing was found.
Maintenance performed	Commands run, why they were chosen, and their results.
Query performance analysis	Execution plan, interpretation, and any justified recommendation for each project.
Recommendations	Prioritized actions for each project: immediate, follow-up, or none.
Limitations and permissions	Permission or logging limitations met, and how they relate to the Activity 4 roles.
Feedback to owners	Findings and recommendations sent to each owning group.
AI usage declaration	AI tools used, tasks performed with them, and how the output was validated.

The PDF must not contain real passwords or other secrets. Check screenshots and logs to confirm that no credentials are visible.

IV. Required Deliverables

- **Health and activity inspection evidence —** For each project database, command output or screenshots with interpretation.
- **Log review notes and issue assessment —** Findings with evidence, explanation, impact, and next step, or a statement of what was checked.
- **Maintenance script or command list —** Commands run with explanations and results, stored in the group's own repository.
- **Query performance analysis —** Execution-plan evidence and interpretation for a selected query in each project.
- **Final maintenance and monitoring report —** A report for each project, compiled into the PDF progress documentation.
- **Progress documentation (PDF) —** A PDF recording all of the group's Activity 5 progress, submitted with the repository link and recorded in the Google Sheets tracker.
- **AI usage declaration —** AI tools used and how generated suggestions were validated.

V. Assessment Rubric

Criteria	Points
Database health and activity inspection	20
Log review and issue interpretation	15
Appropriate maintenance operations and explanation	20
Query execution-plan analysis and recommendations	20

Criteria	Points
Evidence, PDF progress documentation, and actionable recommendations	15
Own repository, tracker update, and version control practices	5
AI transparency and validation	5
Total	**100**

Credit depends on evidence from the real databases and on the commit history in the group's own repository, not on the number of commands run.

VI. Submission Checklist

- ☐ Every other group's project and database were identified from the tracker.
- ☐ The roles and accounts used match the Activity 4 access-control plan and the Activity 3 user roles documentation.
- ☐ Health and activity checks include evidence and interpretation.
- ☐ Available logs were reviewed and findings are grounded in evidence.
- ☐ Maintenance commands are appropriate, explained, and documented.
- ☐ A query execution plan is included and interpreted.
- ☐ Recommendations are prioritized and justified.
- ☐ The scripts and evidence are in the group's own repository, and the Activity 5 Log in the tracker is updated.
- ☐ The PDF progress documentation is complete and contains no secrets.
- ☐ No real credentials or secrets appear in submitted files.
- ☐ AI usage is disclosed and generated suggestions were validated.

Important Notes

> The goal is not to produce the largest number of commands. The goal is to demonstrate sound DBA judgment: observe the database, verify evidence, explain what it means, choose a proportionate action, verify the result, and document what should happen next. Document every project the same way so that the group's PDF progress documentation, repository, and tracker entries show what was observed, what was done, and what should happen next.